EHU

WEB SISTEMAK · BACK-END

Bideak eta txantiloiak

Router, Generator eta EJS

2026 / 27 IKASTURTEA

Itziar Aldabe & Nagore Barrena
Lengoaia eta Sistema Informatikoak

Aurkibidea

01 Bideak

- Metodoak eta erantzunak
- `express.Router()`
- `mergeParams`

02 Express Generator

- Instalazioa eta egitura
- `bin/www` eta `app.js`
- `routes/` karpeta

03 EJS txantiloiak

- `res.render()` eta etiketak
- Begiztak eta baldintzak
- Formularioak: POST
- Fitxategi partzialak

Adibide bera saio osoan zehar: **jabeak eta maskotak**. Bide bakoitzean zati bat gehituko diogu.

Gaurko adibidea: jabeak eta maskotak

```
// data.js
const owners = [
  { id: 1, name: 'Alice', email: 'alice@mail.com' },
  { id: 2, name: 'Bob', email: 'bob@mail.com' },
  { id: 3, name: 'Ane', email: 'ane@mail.com' }
];
const pets = [
  { id: 1, name: 'Rex', species: 'dog', ownerId: 1 },
  { id: 2, name: 'Mimi', species: 'cat', ownerId: 2 },
  { id: 3, name: 'Luna', species: 'dog', ownerId: 1 }
];
module.exports = { owners, pets };
```

Maskota bakoitza `ownerId` bidez lotzen da bere jabearekin. Anek ez du maskotarik.

Gaur ikusiko ditugun bideak

`GET /`

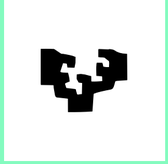
Jabeen zerrenda
→ `views/index.ejs`

`GET /owner/:id`

Jabe baten fitxa eta haren maskotak
→ `views/owner.ejs`

`POST /owner/:ownerId/pets`

Maskota berria gehitu (formularioa)
→ `mergeParams + redirect`



Bideak

Routing eta express.Router()

BIDEAK

Gogoratu: metodoa + bidea → funtzioa

Express-ek eskaera bakoitza **metodoaren** eta **bidearen** arabera bideratzen du: bat datorren lehen bideak erantzuten du.

Metodoa		Bidea		Funtzioa
`app.get`	+	`'/owner/:id'`	→	`(req, res) => { … }`

GET
Informazioa eskatu

POST
Baliabide berria sortu

PUT
Dagoena eguneratu

DELETE
Baliabidea ezabatu

`app.get('/owner/:id', …)` → `GET /owner/1` eskaerari erantzuten dio, baina ez `POST /owner/1`-i.

BIDEAK

Gogoratu: non doa informazioa?

`req.params`

Bidearen segmentuak

Adibidea

```
GET /owner/1
```

Irakurtzeko

```
req.params.id → '1'
```

`req.query`

? ondorengo query string-a

Adibidea

```
GET /owner?name=Alice
```

Irakurtzeko

```
req.query.name → 'Alice'
```

`req.body`

Eskaeraren gorputza (POST, PUT)

Adibidea

```
name=Kira&species=cat
```

Irakurtzeko

```
req.body.name → 'Kira'
```

ADI! `req.params` eta `req.query`-ko balioak beti **testua** dira: `parseInt(req.params.id)`

BIDEAK

Erantzun-metodoak: res

Bide bakoitzak erantzun bat bidali behar du, eta horrela amaitzen da eskaera-erantzun zikloa.

Metodoa	Zer egiten du	Adibidea
`res.send()`	Edozein motatako erantzuna (testua, HTML...)	`res.send('Kaixo')`
`res.json()`	JSON erantzuna: APIetarako	`res.json(owners)`
`res.render()`	Txantiloi bat errendatu eta HTMLa bidali	`res.render('owner', { owner })`
`res.redirect()`	Beste URL batera birbideratu (POST ondoren)	`res.redirect('/owner/1')`
`res.status()`	Egoera-kodea ezarri (kateatu egiten da)	`res.status(404).send('Ez dago')`

res.render() → 3. zatian (EJS)

ADI! Erantzun bakarra eskaera bakoitzeko. Erantzunik ez → eskaera zain geratzen da. Bi erantzun → errorea.

BIDEAK

Arazoa: app.js hazten doa

```
// app.js — dena fitxategi bakarrean
app.get('/', …);
app.get('/owner/:id', …);
app.post('/owner', …);
app.put('/owner/:id', …);
app.delete('/owner/:id', …);
app.get('/owner/:ownerId/pets', …);
app.post('/owner/:ownerId/pets', …);
app.delete('/owner/:ownerId/pets/:petId', …);
// … eta beste 30 bide
app.listen(3000);
```

1. Bide guztiak fitxategi bakar batean
2. `/owner` aurrizkia behin eta berriz errepikatzen da
3. Zaila irakurtzen, mantentzen eta taldean lantzen

Eranskina: `app.route()` →

Irtenbidea

Bideak gaika banatu: modulu bakoitza bere fitxategian → `express.Router()`

BIDEAK

express.Router(): mini-aplikazio bat

```
// routes/owner.js
const express = require('express');
const router = express.Router();                    1
const { owners } = require('../data');

router.get('/:id', (req, res) => { … });            2
router.put('/:id', (req, res) => { … });
router.delete('/:id', (req, res) => { … });

module.exports = router;                            3
```

1. Router instantzia bat sortu: bere bide eta middleware-ak dituen **mini-aplikazioa**
2. `app`-en metodo berberak. Bideak **aurrizkirik gabe**: `'/:id'`
3. Esportatu, `app.js`-n muntatu ahal izateko

BIDEAK

Router-a muntatu: app.use(aurrizkia, router)

```
// app.js
const ownerRouter = require('./routes/owner');
app.use('/owner', ownerRouter);
```

app.js		routes/owner.js		Emaitza
`app.use('/owner', …)`	+	`router.get('/:id', …)`	=	`GET /owner/1`

Router-ean	Benetako URLa
`router.get('/')`	`GET /owner`
`router.get('/:id')`	`GET /owner/1`
`router.put('/:id')`	`PUT /owner/1`
`router.delete('/:id')`	`DELETE /owner/1`

BIDEAK

Gurasoaren parametroak: mergeParams

Aurrizkiak parametro bat badu (`:ownerId`), router-ak ez du ikusten... `mergeParams` aktibatu ezean.

app.js

```
app.use('/owner/:ownerId/pets', petsRouter);                    1
```

routes/pets.js

```
const router = express.Router({ mergeParams: true });           2

router.post('/', (req, res) => {
  const ownerId = parseInt(req.params.ownerId);
  const newPet = { id: pets.length + 1, ownerId,
    name: req.body.name, species: req.body.species };
  pets.push(newPet);
  res.redirect(`/owner/${ownerId}`);
});
```

`POST /owner/3/pets`

mergeParams gabe

`req.params.ownerId` → `undefined`
`ownerId` → `NaN`
→ `/owner/NaN`: 404

mergeParams-ekin

`req.params.ownerId` → `'3'`
→ `/owner/3`: Ane-ren maskota berria



Generator

Express aplikazio baten egitura

GENERATOR

Zer da Express Generator?

Express aplikazio baten oinarrizko egitura **automatikoki** sortzen duen tresna.

Orain arte: eskuz

- `app.js` bakarra: zerbitzaria, bideak eta konfigurazioa nahastuta
- Karpeta-egitura geuk asmatu
- Middleware-ak eskuz gehitu: `express.json()`...

Express Generator-ekin

- Egitura estandarra komando bakarrarekin
- Bideak `routes/` karpetan: **Router-ak** (1. zatia)
- Bistak `views/` karpetan: **txantiloiak** (3. zatia)
- Zerbitzaria `bin/www` fitxategian

Eskakizunak: **Node.js** eta **npm**

GENERATOR

Instalazioa eta proiektua sortzea

1. Instalazioa (globala)

```
> npm install -g express-generator
```

2. Proiektua sortu

```
> express --view=ejs jabeak-maskotak
```

ADI!
`--view` gabe, **jade** erabiltzen du lehenetsita (zaharkitua).
Beti `--view=ejs`.

Aukera	Zertarako
`--view=ejs`	Txantiloi-motorra aukeratu
`--no-view`	Txantiloirik gabe (APIak)
`--git`	.gitignore fitxategia gehitu
`--force`	Karpeta hutsik ez badago ere sortu

GENERATOR

Sortutako proiektuaren egitura

```
jabeak-maskotak/
├── bin/
│   └── www
├── public/
│   ├── images/
│   ├── javascripts/
│   └── stylesheets/
├── routes/
│   ├── index.js
│   └── users.js
├── views/
│   ├── error.ejs
│   └── index.ejs
├── app.js
└── package.json
```

bin/www	Zerbitzaria abiarazten du
public/	Fitxategi estatikoak: CSS, JS, irudiak
routes/	Router-ak: bideen definizioa (1. zatia)
views/	EJS txantiloiak (3. zatia)
app.js	Aplikazioaren konfigurazio nagusia
package.json	Mendekotasunak eta scriptak

GENERATOR

Mendekotasunak instalatu eta exekutatu

```
> cd jabeak-maskotak
> npm install                    1
> npm start                      2
```

package.json

```
"scripts": {
  "start": "node ./bin/www"
}
```

1. Mendekotasunak instalatu → `node_modules/`
2. `start` scripta exekutatu → `./bin/www`
3. Nabigatzailean ireki: `http://localhost:3000`

ADI!
Ez dago berrabiarazte automatikorik: aldaketa bakoitzean zerbitzaria gelditu (Ctrl+C) eta berriro abiarazi.

GENERATOR

nodemon: berrabiarazte automatikoa

```
> npm install --save-dev nodemon
```

package.json

```
"scripts": {
  "start": "node ./bin/www",
  "dev": "nodemon ./bin/www"
}
```

```
> npm run dev
```

npm start

Modu estandarra. Berrabiarazte automatikorik ez.

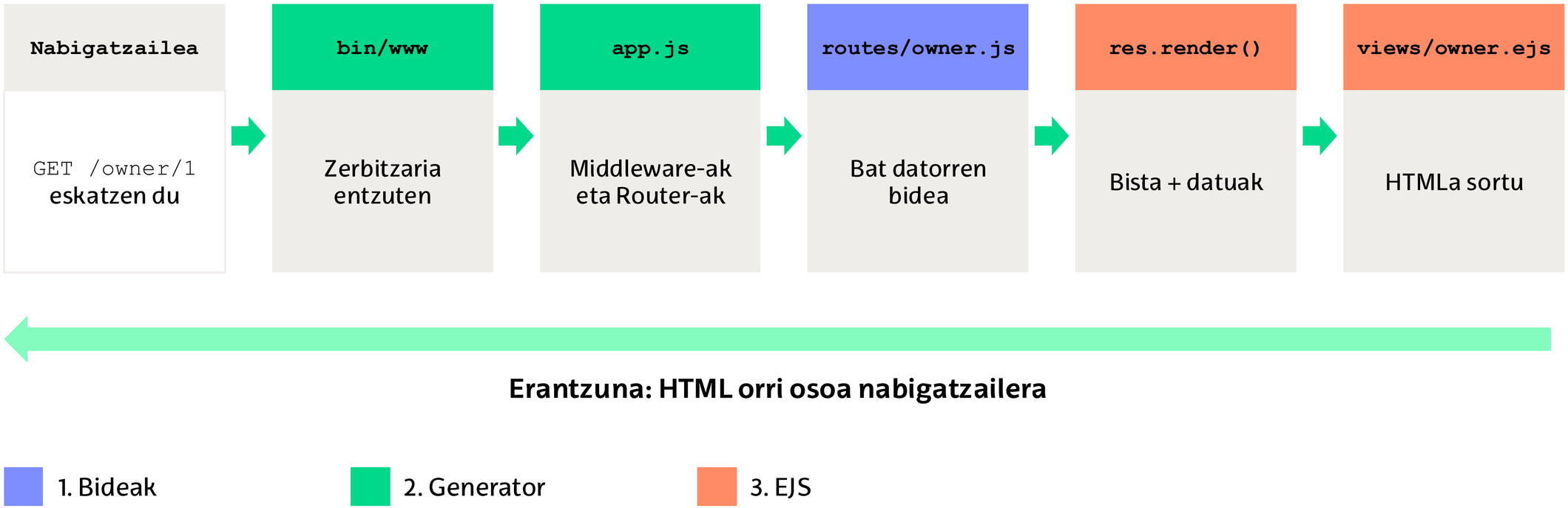
npm run dev

Garapena: fitxategi bat aldatzean zerbitzaria automatikoki berrabiarazten da.

GENERATOR

Eskaera baten bidaia

Zer gertatzen da proiektuan nabigatzaileak `/owner/1` eskatzen duenean?



GENERATOR

bin/www: zerbitzariaren sarrera-puntua

Eskuz: `app.listen(3000)`. Generator-ekin: gauza bera, baina fitxategi bereizi batean.

```
#!/usr/bin/env node
var app = require('../app');                                    1
var http = require('http');

var port = normalizePort(process.env.PORT || '3000');           2
app.set('port', port);

var server = http.createServer(app);                            3
server.listen(port);
server.on('error', onError);                                    4
server.on('listening', onListening);
```

1. `app.js` inportatu: aplikazioaren konfigurazio osoa
2. Ataka: `PORT` aldagaia edo 3000
3. HTTP zerbitzaria sortu eta entzuten jarri
4. Abiaraztean erroreak eta gertaerak kudeatu

GENERATOR

app.js: dena lotzen duen fitxategia

```
var express = require('express');
var path = require('path');
var indexRouter = require('./routes/index');                    1
var ownerRouter = require('./routes/owner');
var petsRouter = require('./routes/pets');
var app = express();

app.set('views', path.join(__dirname, 'views'));                2
app.set('view engine', 'ejs');

app.use(express.json());                                        3
app.use(express.urlencoded({ extended: false }));
app.use(express.static(path.join(__dirname, 'public')));

app.use('/', indexRouter);                                      4
app.use('/owner', ownerRouter);
app.use('/owner/:ownerId/pets', petsRouter);
// + logger, cookieParser, 404 eta erroreak
module.exports = app;
```

1. Router-ak inportatu
2. EJS konfiguratuta dago **jada**
3. HTTP klaseko middleware-ak: hemen daude jada!
4. Router-ak aurrizki banatan muntatu (1. zatia)

GENERATOR

routes/index.js

Generator-ek sortua

```
router.get('/', function(req, res, next) {
  res.render('index', { title: 'Express' });
});
```

Guk egokitua

```
var express = require('express');
var router = express.Router();                    1
const { owners } = require('../data');            2

router.get('/', function(req, res, next) {
  res.render('index', { owners });                3
});

module.exports = router;
```

1. 1. zatiko **Router** bera, bere fitxategian
2. Datuak inportatu: jabeen zerrenda
3. `res.render()`: bista + datuak → 3. zatia

GENERATOR

routes/owner.js: URL parametroak

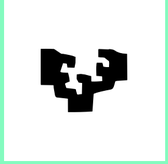
`GET /owner/1` = `app.use('/owner', …)` + `router.get('/:id', …)`

```
const express = require('express');
const router = express.Router();
const { owners, pets } = require('../data');

router.get('/:id', (req, res, next) => {
  const id = parseInt(req.params.id);                    1
  const owner = owners.find(o => o.id === id);           2
  if (!owner) return next(); // 404
  const ownerPets = pets.filter(p => p.ownerId === id);  3
  res.render('owner', { owner, ownerPets });             4
});

module.exports = router;
```

1. Parametroa testua da → `parseInt()`
2. Jabea bilatu; ez badago, `next()` → 404
3. Jabe horren maskotak
4. Bista errendatu bi datuekin



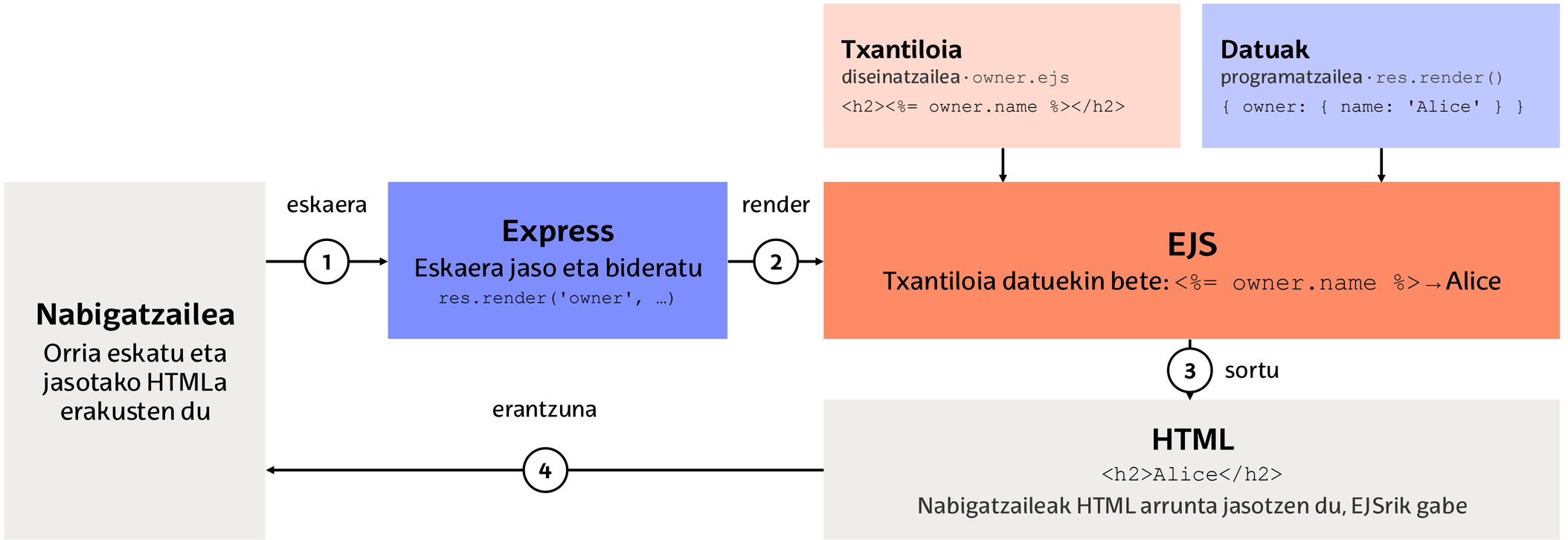
Txantiloiak

EJS: datuak HTML bihurtu

EJS

Txantiloiak: nola funtzionatzen du EJS-k?

EJS (Embedded JavaScript): Node.js-erako txantiloi-motorra. **Txantiloia** (itxura) eta **datuak** (edukia) bereizita daude; EJS-k elkartzen ditu HTML bihurtzeko.



EJS

res.render(): bista bat errendatzea

```
res.render('owner', …)
```

1. Bistaren izena, luzapenik gabe → `views/owner.ejs`

```
res.render(…, { owner, ownerPets })
```

2. Datuak: gako bakoitza **aldagai** bat da bistan

```
res.render('pets/list')
```

3. Azpikarpetak: → `views/pets/list.ejs`

Konfigurazioa `app.js`-n dago jada. Generator gabe: `npm install ejs` + `app.set('view engine', 'ejs')`

EJS

EJS etiketak

Etiketa	Zer egiten du	Adibidea
`<%= %>`	Balioa idazten du, HTMLa **ihes eginda**	`<%= owner.name %>`
`<%- %>`	Balioa idazten du, ihes egin **gabe**	`<%- include('partials/header') %>`
`<% %>`	JS kodea exekutatzen du; ez du ezer idazten	`<% if (ownerPets.length) { %>`
`<%# %>`	Iruzkina: ez da exekutatzen ez idazten	`<%# Jabeen zerrenda %>`

ADI!
Erabiltzaileen datuak beti `<%= %>`-rekin. `<%- %>`-rekin `<script>` bat txerta liteke (XSS).

EJS

Datuak bistara pasatzea

```
// routes/owner.js
res.render('owner', { owner, ownerPets });
```

```
<!-- views/owner.ejs -->
<h2>Jabea: <%= owner.name %></h2>
<p>Email: <%= owner.email %></p>
```

```
<!-- nabigatzaileak jasotzen duena -->
<h2>Jabea: Alice</h2>
<p>Email: alice@mail.com</p>
```

1. Objektuaren gakoak → bistako aldagaiak
2. Propietateetara sarbidea: `owner.name`, `owner.email`
3. HTMLa beteta iristen da: ez dago EJSrik

EJS

Zerrendak: begiztak

```
res.render('index', { owners });
```

```
<ul>
  <% owners.forEach(o => { %>
    <li><a href="/owner/<%= o.id %>"><%= o.name %></a> - <%= o.email %></li>
  <% }) %>
</ul>
<p>Jabeak guztira: <%= owners.length %></p>
```

```
<li><a href="/owner/1">Alice</a> - alice@mail.com</li>
<li><a href="/owner/2">Bob</a> - bob@mail.com</li>
<li><a href="/owner/3">Ane</a> - ane@mail.com</li>
<p>Jabeak guztira: 3</p>
```

1. Jabeen **array**-a pasatzen dugu
2. `<% %>` exekutatu (begizta)
   `<%= %>` balioa idatzi
3. Jabe bakoitzeko `<li>` bat; estekak → `/owner/:id`

EJS

Baldintzak

views/owner.ejs

```
<% if (ownerPets.length === 0) { %>                                    1
  <p>Jabe honek ez du erregistratutako maskotarik.</p>
<% } else { %>                                                         2
  <table>
    <tr><th>ID</th><th>Izena</th><th>Espeziea</th></tr>
    <% ownerPets.forEach(pet => { %>
      <tr>
        <td><%= pet.id %></td>
        <td><%= pet.name %></td>
        <td><%= pet.species %></td>
      </tr>
    <% }) %>
  </table>
<% } %>
```

1 · `/owner/3` (Ane)

Jabe honek ez du erregistratutako maskotarik.

2 · `/owner/1` (Alice)

ID	Izena	Espeziea
1	Rex	dog
3	Luna	dog

EJS

Formularioak: POST eta redirect

views/owner.ejs

```
<form action="/owner/<%= owner.id %>/pets" method="POST">
  <input type="text" name="name" placeholder="Maskota izena" required>
  <input type="text" name="species" placeholder="Espeziea" required>
  <button type="submit">Gehitu</button>
</form>
```

1 Nabigatzailea

`POST /owner/3/pets`
name, species

→

2 routes/pets.js

`req.params.ownerId`
+ `req.body` → gorde

→

3 res.redirect()

`'/owner/3'`
Ez du bistarik errendatzen

→

4 Nabigatzailea

`GET /owner/3`
Maskota berria taulan

Zergatik redirect? Orria freskatzean (F5) formularioa **ez da berriro bidaltzen**: POST / Redirect / GET

EJS

Fitxategi partzialak: include

```
<!-- views/partials/header.ejs -->
<!DOCTYPE html> … <body>
<header><h1>Jabe eta maskoten sistema</h1> … </header>
```

```
<!-- views/index.ejs -->
<%- include('partials/header') %>
<h2>Jabe zerrenda</h2> …
<%- include('partials/footer') %>
```

```
<!-- views/partials/footer.ejs -->
<footer><p>&copy; 2026 Infor albaitaritza</p></footer>
</body></html>
```

1. Zati berrerabilgarria, `views/partials/`-en
2. `include`-k zatia txertatzen du `<%-`: HTMLa ihes egin gabe
3. Goiburu eta orri-oin bera orri guztietan

Laburpena: GET /owner/1 hasieratik bukaerara

Nabigatzailea	`GET http://localhost:3000/owner/1`	Eskaera
bin/www	`server.listen(port)`	Zerbitzaria entzuten
app.js	`app.use('/owner', ownerRouter)`	Aurrizkia → Router
routes/owner.js	`router.get('/:id', …)`	Bidea + datuak bilatu
res.render()	`res.render('owner', { owner, ownerPets })`	Bista + datuak
views/owner.ejs	`<h2><%= owner.name %></h2>`	HTMLa sortu → nabigatzailera

Ariketa

Oinarrizkoa

`index.ejs`-n, jabe bakoitzaren ondoan haren **maskota kopurua** erakutsi.

Pista
Non kalkulatu: bidean ala bistan?

Ertaina

`GET /pets` bide berria: maskota guztiak, **jabearen izenarekin**. Bista berria: `pets.ejs`.

Pista
`find()` + partzialak berrerabili

Hedapena

Maskota bat **ezabatu**: botoi bat taulako lerro bakoitzean.

Pista
`POST …/pets/:petId/delete` + redirect

Baliabideak

Repoa: `jabeak-maskotak`

`01-ejs-oinarria/`

EJS bakarrik, Generator gabe
`app.js` bakarra: bideak, datuak eta zerbitzaria

`02-generator/`

Gaurko klasearen egitura
`bin/www`, `app.js`, `data.js`, `routes/`, `views/`

```
> cd 02-generator
> npm install
> npm run dev     # localhost:3000
```

Dokumentazioa

Routing
`expressjs.com/en/guide/routing.html`

Express Generator
`expressjs.com/en/starter/generator.html`

EJS
`ejs.co`

ERANSKINA

Eranskina: app.route()

Bide bera metodo desberdinekin: bidea **behin bakarrik** idatzi eta metodoak kateatu.

Orain arte

```
app.get('/owner/:id', …);
app.put('/owner/:id', …);
app.delete('/owner/:id', …);
```

app.route()-rekin

```
app.route('/owner/:id')                    1
   .get((req, res) => { … })               2
   .put((req, res) => { … })
   .delete((req, res) => { … });
```

Router-ekin ere bai

```
router.route('/:id')                       3
   .get(…)
   .put(…)
   .delete(…);
```

1. Bidea behin bakarrik idazten da
2. Metodoak kateatuta, puntuz puntu
3. Router baten barruan: aurrizkirik gabe, ohi bezala

Ez ditu fitxategiak banatzen: horretarako `express.Router()`.

← 8. diapositibara itzuli

EHU

WEB SISTEMAK · BACK-END

Bideak eta txantiloiak

Router, Generator eta EJS

2026 / 27 IKASTURTEA

Itziar Aldabe & Nagore Barrena
Lengoaia eta Sistema Informatikoak